

Міністерство освіти і науки України
Національний університет «Запорізька політехніка»

кафедра програмних засобів

ЗВІТ

з лабораторної роботи № 3

з дисципліни «Технології розподілених систем та паралельних
обчислень» на тему:

**«ВИКОРИСТАННЯ БІБЛІОТЕКИ MPI ДЛЯ СТВОРЕННЯ
ПАРАЛЕЛЬНИХ ПРОГРАМ»**

Виконав:

ст. гр. КНТ-113сп

Іван ЩЕДРОВСЬКИЙ

Прийняв:

Старший викладач

Олександр КАЧАН

1 Мета роботи

Вивчити каркас паралельних програм на базі функцій бібліотеки стандарту MPI, навчитися створювати паралельні програми та запускати їх на виконання

2 Завдання до лабораторної роботи

2.1 Змінити вихідну програму 1 так, щоб процес з номером, що дорівнює номеру робочого місця, виводив прізвище студента, а інші процеси виводили свій порядковий номер.

2.2 Доповніть вихідну програму 2 так, щоб процес "1" виконував посилку, а процес "0" – прийом елементів масиву чисел типу float, причому розміри буферів передачі та прийому мають дорівнювати номеру робочого місця, помноженому на 11, а кількість елементів, що передаються, дорівнює номеру робочого місця, збільшеному на одиницю. В процесі "0" вивести кількість фактично прийнятих елементів.

2.3 Переробити програму завдання 2 так, щоб в якості буферів прийому та передачі використовувались масиви динамічної пам'яті.

2.4 Напишіть програму, яка складається з чотирьох процесів. Процес "0" передає до процесів "1", "2" та "3" рядок з прізвищем студента. Процес "1" конкатенує прийнятий рядок з рядком, відповідним імені студента, і відсилає отриманий рядок назад. Процес "2" визначає кількість символів в прийнятому рядку та відсилає це число до нульового процесу. Процес "3" множить число, що дорівнює сумі кодів символів прийнятого рядка на число, та відсилає отримане значення до процесу "0". Після завершення обмінів процес "0" виводить на друк отримані від інших процесів значення

3 Виконання лабораторної роботи

Для виконання першого завдання було взято вихідний код програми один який складався з ініціалізації MPI, взятті рангу та кількості процесів, їх показу, показу аргументів командного рядку та закриттю MPI. Для початку цей код був переписаний, з нього було видалено коментарі, змінено назви змінних. Було видалено показ аргументів командного рядку, оскільки MPI автоматично обрізає свої внутрішні параметри, це причина чому ми передаємо `argc` та `argv` як вказівники, тому в цих аргументах будуть лише аргументи, які ми передаємо. Далі було переписано показ інформації згідного мого варіанту(30).

Результат виконання першого завдання показаний на рисунку 1.

```
C:\home\university-4\distributed-systems\lb3\x64\Debug>mpiexec -n 31 lb3.exe
4 (p=31): 4
8 (p=31): 8
20 (p=31): 20
24 (p=31): 24
1 (p=31): 1
7 (p=31): 7
5 (p=31): 5
21 (p=31): 21
28 (p=31): 28
29 (p=31): 29
26 (p=31): 26
15 (p=31): 15
18 (p=31): 18
27 (p=31): 27
30 (p=31): Schedrovskyi Ivan
16 (p=31): 16
25 (p=31): 25
23 (p=31): 23
9 (p=31): 9
17 (p=31): 17
22 (p=31): 22
6 (p=31): 6
0 (p=31): 0
19 (p=31): 19
11 (p=31): 11
14 (p=31): 14
10 (p=31): 10
13 (p=31): 13
3 (p=31): 3
2 (p=31): 2
12 (p=31): 12
```

Рисунок 1 – Виконання завдання 1

Для виконання другого завдання було взято готовий код та також виконано його рефакторинг, для подальшої зручної роботи з ним. Оригінальний код відправляє пустий масив double на 20 елементів через MPI, передаючи тільки 5 з процесу 0 на 1, а потім процес 1 їх приймає.

Далі в другому завданні було виконано завдання згідно з варіантом 30. Створено буфер на 330 елементів, передано 31 float елемент з 1 в 0.

Результат виконання завдання 2 показаний на рисунку 2.

```
C:\home\university-4\distributed-systems\lb3\x64\Debug>mpiexec -n 2 lb3.exe
0: hello (p = 2)
0: Sending 5 elements to 1
0: Received 31 elements from 1
0: bye
1: hello (p = 2)
1: Received 5 elements from 0
1: Sending 31 float element to 0
1: bye
```

Рисунок 2 – Виконання завдання 2

Для виконання завдання 3 було взято код завдання 2 та в ньому змінено логіку створення масивів з статичних на динамічні через `new double[20]` та `new float`.

Результат виконання завдання 3 показаний на рисунку 3.

```
C:\home\university-4\distributed-systems\lb3\x64\Debug>mpiexec -n 2 lb3.exe
1: hello (p = 2)
1: Received 5 elements from 0
1: Sending 31 float element to 0
1: bye
0: hello (p = 2)
0: Sending 5 elements to 1
0: Received 31 elements from 1
0: bye
```

Рисунок 3 – Виконання завдання 3

Для виконання четвертого завдання було написано програму яка працює з чотирма процесами. Нульовий спочатку надсилає своє прізвище до інших трьох процесів, а потім показує результати їх роботи. Оскільки робота ведеться в C++, то для збереження інформації було використано `std::string`, а для їх коректного прийняття в `char*` було використано функцію `MPI_Probe`, яка перед отриманням результату дозволяє створити буфер потрібного розміру.

Результат виконання завдання 4 показаний на рисунку 4.

```

C:\home\university-4\distributed-systems\lb3\x64\Debug>mpiexec -n 4 lb3.exe
1: hello (p = 4)
1: Received surname from 0
1: Send full name to 0
1: bye
0: hello (p = 4)
0: Send name to 1
0: Send name to 2
0: Send name to 3
0: Received full name from 1
0: Full name from 1: Shchedrovskyi Ivan
0: Characters from 2: 13
0: Sum from 3: 4303.981935
0: bye
3: hello (p = 4)
3: Received surname from 0
3: Send 4303.981935 to 0
3: bye
2: hello (p = 4)
2: Received surname from 0
2: Send 13 to 0
2: bye

```

Рисунок 4 – Виконання завдання 4

4 Тест розробленої програми

```

#include <iostream>
#include "mpi.h"
# define M_PI          3.14159265358979323846  /* pi */

void a(int argc, char** argv) {
    MPI_Init(&argc, &argv);

    int number_process;
    int rank;

    MPI_Comm_size(MPI_COMM_WORLD, &number_process);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    printf("%d: hello (p = %d)", rank, number_process);

    MPI_Finalize();
}

void first(int argc, char** argv) {
    int number_process;
    int rank;

    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &number_process);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    // printf("%d: hello (p = %d)\n", rank, number_process);

```

```

    if (rank == 30) {
        printf("%2d (p=%d): Schedrovskiy Ivan\n", rank, number_process);
    } else {
        printf("%2d (p=%d): %d\n", rank, number_process, rank);
    }

    // printf("%d: bye\n", rank);
    MPI_Finalize();
}

void second(int argc, char* argv[]) {
    int number_process;
    int rank;

    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &number_process);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    printf("%d: hello (p = %d)\n", rank, number_process);

    if (number_process != 2) {
        printf("%d: Only 2 processes required, but p = %d\n", rank, number_process);

        MPI_Barrier(MPI_COMM_WORLD);
        MPI_Abort(MPI_COMM_WORLD, MPI_ERR_OTHER);
        return;
    }

    if (rank == 0) {
        double doubleNumbers[20];

        printf("%d: Sending 5 elements to 1\n", rank);
        MPI_Send(doubleNumbers, 5, MPI_DOUBLE, 1, 100, MPI_COMM_WORLD);

        MPI_Status status;
        double variantNumbers[30 * 11];

        MPI_Recv(variantNumbers, 31, MPI_FLOAT, 1, 101, MPI_COMM_WORLD, &status);

        int count;
        MPI_Get_count(&status, MPI_FLOAT, &count);
        printf("%d: Received %d elements from %d\n", rank, count, status.MPI_SOURCE);
    }

    if (rank == 1) {
        MPI_Status status;
        double doubleNumbers[20];

        MPI_Recv(doubleNumbers, 5, MPI_DOUBLE, 0, 100, MPI_COMM_WORLD, &status);

        int count;
        MPI_Get_count(&status, MPI_DOUBLE, &count);
        printf("%d: Received %d elements from %d\n", rank, count, status.MPI_SOURCE);

        float variantNumbers[30 * 11];
        printf("%d: Sending 31 float element to 0\n", rank);
        MPI_Send(variantNumbers, 31, MPI_FLOAT, 0, 101, MPI_COMM_WORLD);
    }

    printf("%d: bye\n", rank);
    MPI_Finalize();
}

void third(int argc, char* argv[]) {
    int number_process;

```

```

int rank;

MPI_Init(&argc, &argv);
MPI_Comm_size(MPI_COMM_WORLD, &number_process);
MPI_Comm_rank(MPI_COMM_WORLD, &rank);

printf("%d: hello (p = %d)\n", rank, number_process);

if (number_process != 2) {
    printf("%d: Only 2 processes required, but p = %d\n", rank, number_process);

    MPI_Barrier(MPI_COMM_WORLD);
    MPI_Abort(MPI_COMM_WORLD, MPI_ERR_OTHER);
    return;
}

if (rank == 0) {
    double *doubleNumbers = new double[20];

    printf("%d: Sending 5 elements to 1\n", rank);
    MPI_Send(doubleNumbers, 5, MPI_DOUBLE, 1, 100, MPI_COMM_WORLD);

    MPI_Status status;
    float *variantNumbers = new float[30 * 11];
    MPI_Recv(variantNumbers, 31, MPI_FLOAT, 1, 101, MPI_COMM_WORLD, &status);

    int count;
    MPI_Get_count(&status, MPI_FLOAT, &count);
    printf("%d: Received %d elements from %d\n", rank, count, status.MPI_SOURCE);

    delete[] doubleNumbers;
    delete[] variantNumbers;
}

if (rank == 1) {
    MPI_Status status;
    double* doubleNumbers = new double[20];
    MPI_Recv(doubleNumbers, 5, MPI_DOUBLE, 0, 100, MPI_COMM_WORLD, &status);

    int count;
    MPI_Get_count(&status, MPI_DOUBLE, &count);
    printf("%d: Received %d elements from %d\n", rank, count, status.MPI_SOURCE);

    float* variantNumbers = new float[30 * 11];
    printf("%d: Sending 31 float element to 0\n", rank);
    MPI_Send(variantNumbers, 31, MPI_FLOAT, 0, 101, MPI_COMM_WORLD);

    delete[] doubleNumbers;
    delete[] variantNumbers;
}

printf("%d: bye\n", rank);
MPI_Finalize();
}

void four(int argc, char* argv[]) {
    int number_process;
    int rank;

    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &number_process);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    printf("%d: hello (p = %d)\n", rank, number_process);

```

```

if (number_process != 4) {
    printf("%d: Only 4 processes required, but p = %d\n", rank, number_process);

    MPI_Barrier(MPI_COMM_WORLD);
    MPI_Abort(MPI_COMM_WORLD, MPI_ERR_OTHER);
    return;
}

if (rank == 0) {
    std::string SURNAME = "Shchedrovskiy";

    printf("%d: Send name to 1\n", rank);
    MPI_Send(SURNAME.c_str(), std::size(SURNAME) + 1, MPI_CHAR, 1, 5,
MPI_COMM_WORLD);

    printf("%d: Send name to 2\n", rank);
    MPI_Send(SURNAME.c_str(), std::size(SURNAME) + 1, MPI_CHAR, 2, 5,
MPI_COMM_WORLD);

    printf("%d: Send name to 3\n", rank);
    MPI_Send(SURNAME.c_str(), std::size(SURNAME), MPI_CHAR, 3, 5, MPI_COMM_WORLD);

    MPI_Status status;

    int length;
    MPI_Probe(1, 6, MPI_COMM_WORLD, &status);
    MPI_Get_count(&status, MPI_CHAR, &length);

    printf("%d: Received full name from 1\n", rank);
    char* fullName = new char[length];
    MPI_Recv(fullName, length, MPI_CHAR, 1, 6, MPI_COMM_WORLD, &status);
    printf("%d: Full name from 1: %s\n", rank, fullName);

    delete[] fullName;

    int chars;
    MPI_Recv(&chars, 1, MPI_INT, 2, 7, MPI_COMM_WORLD, &status);
    printf("%d: Characters from 2: %d\n", rank, chars);

    double sum;
    MPI_Recv(&sum, 1, MPI_DOUBLE, 3, 8, MPI_COMM_WORLD, &status);
    printf("%d: Sum from 3: %f\n", rank, sum);

    printf("%d: bye\n", rank);
    MPI_Finalize();
    return;
}

MPI_Status status;
int length;

MPI_Probe(0, 5, MPI_COMM_WORLD, &status);
MPI_Get_count(&status, MPI_CHAR, &length);
printf("%d: Received surname from 0\n", rank);

char* buf = new char[length];
MPI_Recv(buf, length, MPI_CHAR, 0, 5, MPI_COMM_WORLD, &status);
std::string surname = buf;
delete[] buf;

if (rank == 1) {
    std::string NAME = "Ivan";
    std::string FULL_NAME = surname + " " + NAME;

    printf("%d: Send full name to 0\n", rank);

```

```

        MPI_Send(FULL_NAME.c_str(), std::size(FULL_NAME)+1, MPI_CHAR, 0, 6,
MPI_COMM_WORLD);
    }

    if (rank == 2) {
        int chars = surname.length();

        printf("%d: Send %d to 0\n", rank, chars);
        MPI_Send(&chars, 1, MPI_INT, 0, 7, MPI_COMM_WORLD);
    }

    if (rank == 3) {
        double sum = 0;
        for (char c : surname) {
            sum += c;
        }
        double result = sum * M_PI;

        printf("%d: Send %f to 0\n", rank, result);
        MPI_Send(&result, 1, MPI_DOUBLE, 0, 8, MPI_COMM_WORLD);
    }

    printf("%d: bye\n", rank);
    MPI_Finalize();
}

int main(int argc, char** argv) {
    //first(argc, argv);
    //second(argc, argv);
    //third(argc, argv);
    four(argc, argv);

    return 0;
}

```

5 Висновки

Я вивчив каркас паралельних програм на базі функцій бібліотеки стандарту MPI, навчився створювати паралельні програми та запускати їх на виконання